Exposed Metrics (OWASP Juice Shop)

Severity: Low — Context-dependent

1. Concept Overview

Exposed Metrics is an information disclosure issue that occurs when an application's monitoring or metrics endpoint is publicly accessible without appropriate access restrictions.
Metrics are numerical measurements collected to monitor an application's performance, availability, and operational activity.
Applications commonly expose metrics through a /metrics endpoint, which can be accessed by monitoring systems such as Prometheus to collect application statistics.
In OWASP Juice Shop, this challenge requires identifying the endpoint that provides usage data to a monitoring system.

2. Methodology — How I Solved the Lab

Step 1: Identify the target application
Confirmed that OWASP Juice Shop was running locally at: **http://localhost:3000**

Step 2: Discover the metrics endpoint
Added /metrics to the application URL and accessed it through the browser: **http://localhost:3000/metrics**

Step 3: Analyze the exposed information
The endpoint displayed application monitoring metrics in a machine-readable format. These metrics are intended to be collected by monitoring systems such as Prometheus.

Step 4: Verify the challenge
Successfully accessed the metrics endpoint and confirmed that the Exposed Metrics challenge was marked as solved in the OWASP Juice Shop Score Board.

3. Security Impact — How Could It Expose Information?

When metrics endpoints are publicly accessible, unauthorized users may obtain operational information about the application, such as:

- Application request counts and traffic patterns.
- HTTP response statistics and error rates.
- Application performance and resource usage.
- Internal service or endpoint names, depending on the metrics configuration.
- Operational details that could help attackers understand the application's architecture.

An attacker could use this information to identify application behavior, understand potential weaknesses, and plan further attacks.

Important: Accessing /metrics does not automatically expose customer passwords or personal records. The actual information disclosed depends on the metrics collected by the application.

4. Remediation

The following security measures can help prevent unauthorized access to metrics endpoints:

1. **Restrict endpoint access:** Allow only authorized monitoring systems and internal network addresses to access /metrics.
2. **Implement authentication:** Require appropriate authentication or access tokens for metrics collection.
3. **Use network restrictions:** Keep monitoring endpoints private and prevent direct public internet access.
4. **Minimize exposed information:** Avoid including passwords, authentication tokens, personal information, or other sensitive values in metric labels.
5. **Disable unnecessary exposure:** Disable the metrics endpoint when monitoring functionality is not required.

Changing the endpoint URL alone is not an effective security solution. Access control is essential.

5. Evidence to Include

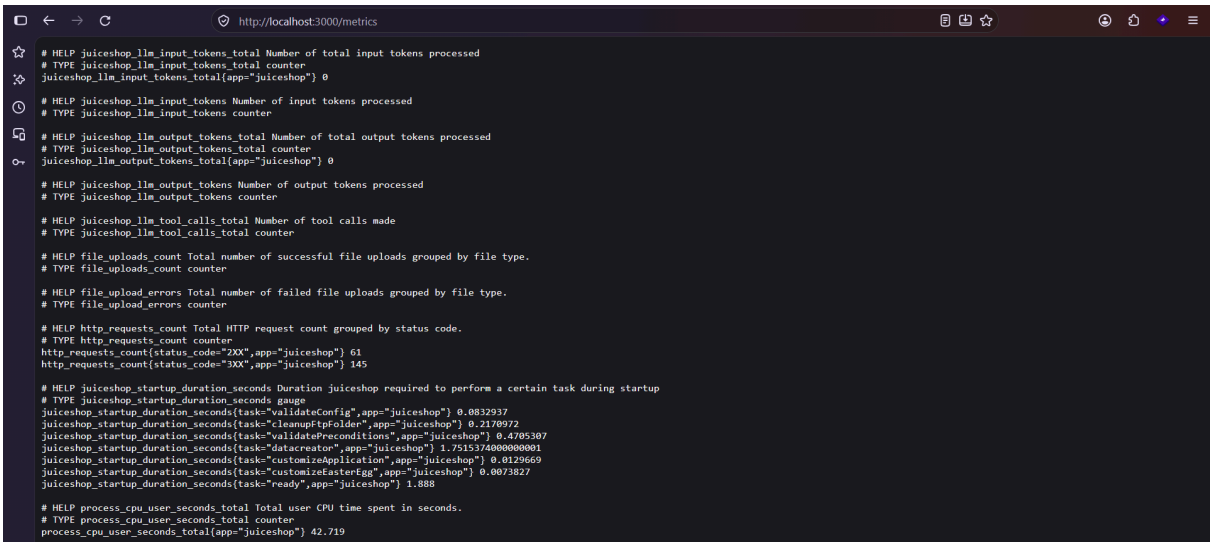


Figure 1: Browser displaying **http://localhost:3000/metrics.**

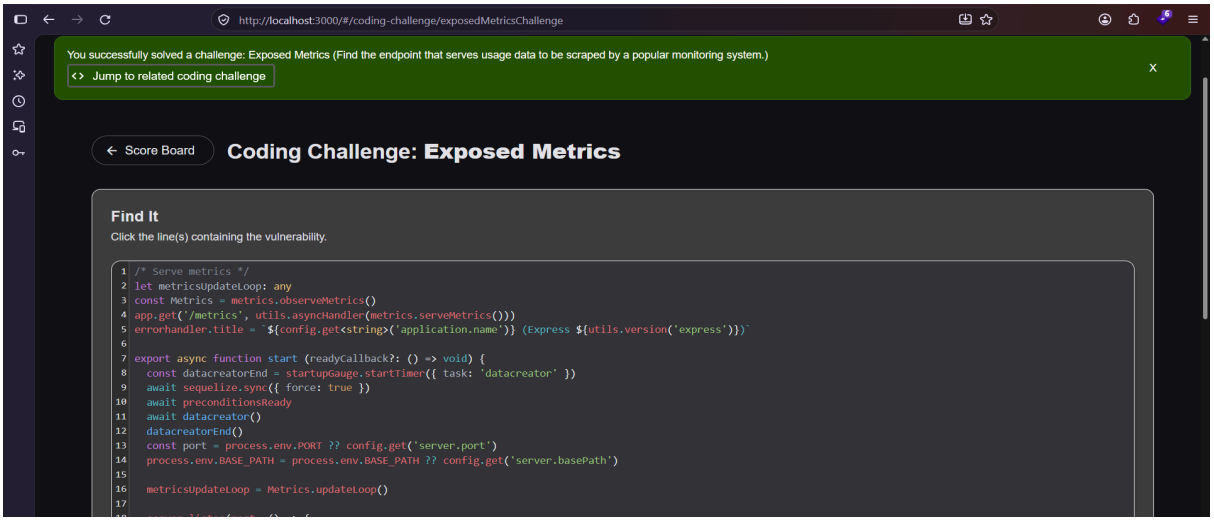


Figure 2: OWASP Juice Shop Score Board showing the Exposed Metrics challenge as completed.

6. Result

Successfully completed the Exposed Metrics challenge in OWASP Juice Shop by discovering and accessing the /metrics endpoint.
This exercise helped me understand application monitoring endpoints, Prometheus metrics collection, information disclosure risks, and the importance of restricting access to internal operational data.
Key Learning: Monitoring endpoints should not be unnecessarily exposed to unauthorized users. Proper authentication, network restrictions, and sensitive-data protection help prevent information disclosure.

References

1. OWASP Juice Shop — Challenge Guide
2. Prometheus — Getting Started

Suggested finding classification: Observability Failures — Exposed Metrics. Severity is Low for this basic lab demonstration, but could be higher in a real application if the endpoint exposes sensitive operational information or enables further attacks.

Vulnerability Identification and Code Remediation

1. About the Vulnerability

Exposed Metrics is an information disclosure vulnerability that occurs when an application's monitoring endpoint is accessible to unauthorized users.
In OWASP Juice Shop, the /metrics endpoint provides application monitoring information, which can be used by monitoring systems such as Prometheus.
If this endpoint is publicly accessible, attackers may gather internal application details, performance statistics, request information, and other operational data.

2. Vulnerability Identification — Find It

The application exposes its metrics through the following Express.js route:
app.get('/metrics', utils.asyncHandler(metrics.serveMetrics()))

Explanation:

- app.get() defines an HTTP GET endpoint.
- /metrics is the URL used to access application metrics.
- metrics.serveMetrics() provides the monitoring data.
- utils.asyncHandler() handles asynchronous execution and errors.

The issue is that the endpoint does not enforce administrator authorization before providing metrics.

3. Security Impact

An unauthorized user accessing the endpoint could potentially discover:

- Application performance and request statistics.
- HTTP request and response information.
- Internal application operational details.

- Service and resource usage information.

This information could help attackers understand the application environment and plan further attacks.

4. Remediation — Fix It

Selected Fix 3 in the OWASP Juice Shop coding challenge.

Vulnerable code:

app.get('/metrics', utils.asyncHandler(metrics.serveMetrics()))

Corrected code:

app.get('/metrics', security.isAdmin(), utils.asyncHandler(metrics.serveMetrics()))

Explanation:
The remediation introduces the security.isAdmin() middleware before the metrics handler.

- security.isAdmin() checks whether the requesting user has administrator permissions.
- Unauthorized users are prevented from accessing the metrics endpoint.
- Authorized administrators can continue accessing the monitoring information.

This ensures that metrics are not publicly accessible to regular users.

5. Result

Successfully completed the Exposed Metrics coding challenge by identifying the publicly accessible /metrics endpoint and selecting Fix 3 to introduce administrator-level authorization.
This exercise improved my understanding of information disclosure, Express.js middleware, access control, and securing application monitoring endpoints.
Key Learning: Sensitive monitoring endpoints should not be publicly accessible. Implementing proper authentication and role-based authorization helps protect internal application information.

6. Evidence

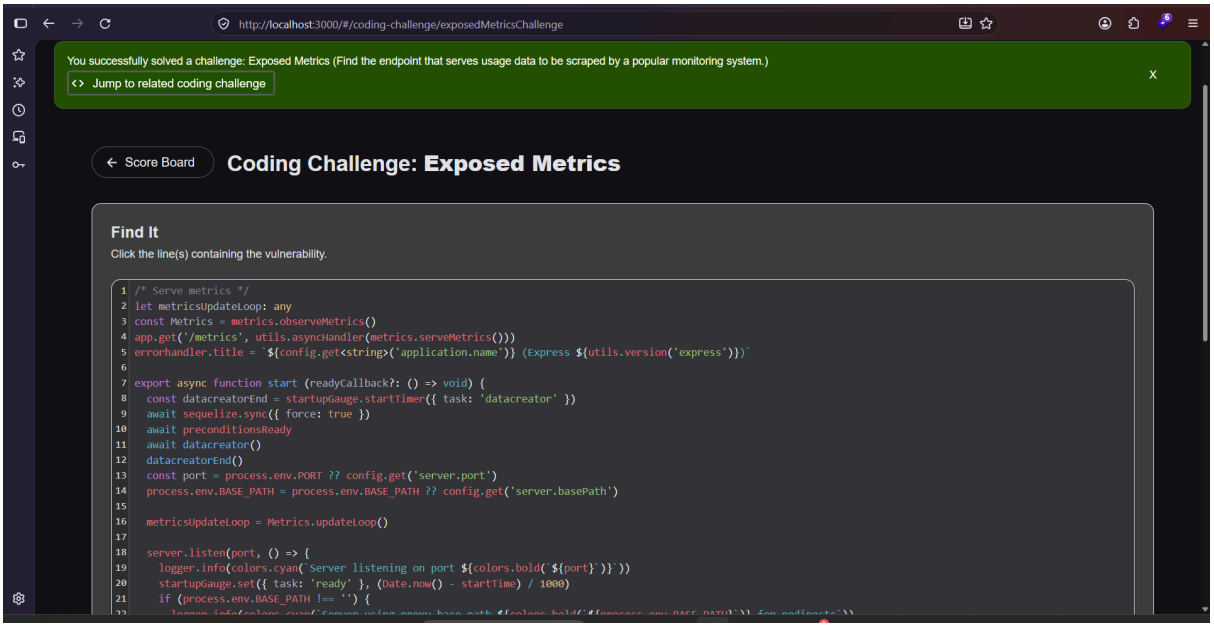


Figure 1: Identified the exposed /metrics endpoint.

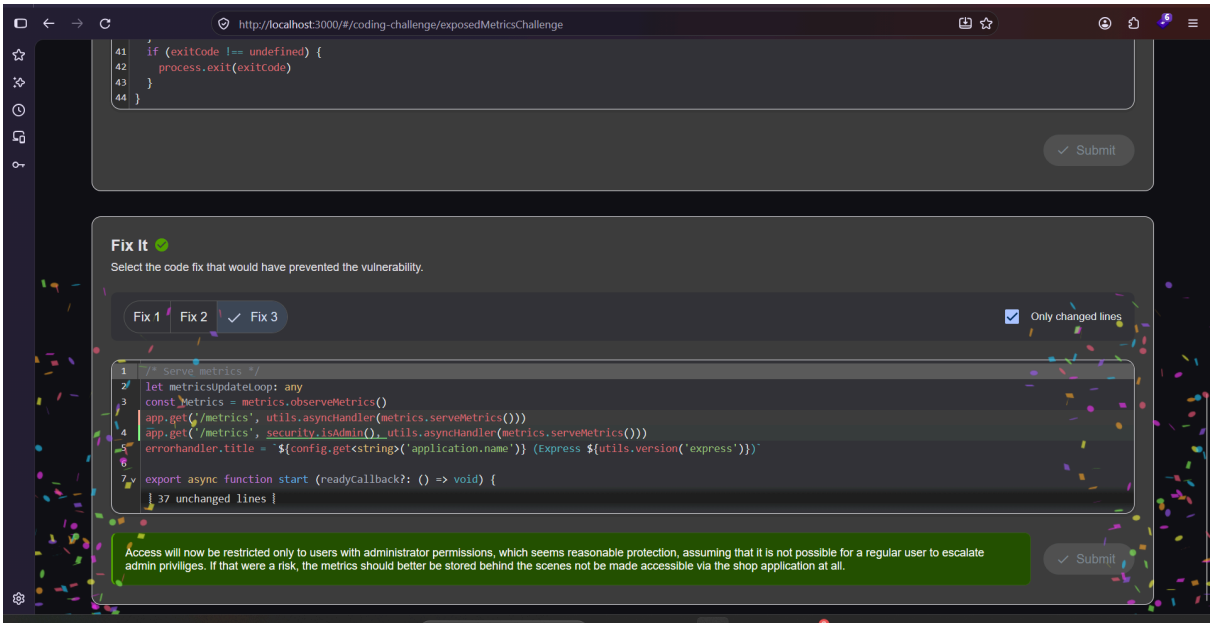


Figure 2: Selected Fix 3 to implement administrator authorization.

Security classification: Information Disclosure / Broken Access Control.

Suggested severity: Low to Medium, depending on the sensitivity of the exposed metrics and the application's environment.